

Aim- Perform geometric transformation.

[illegible]

CODE:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

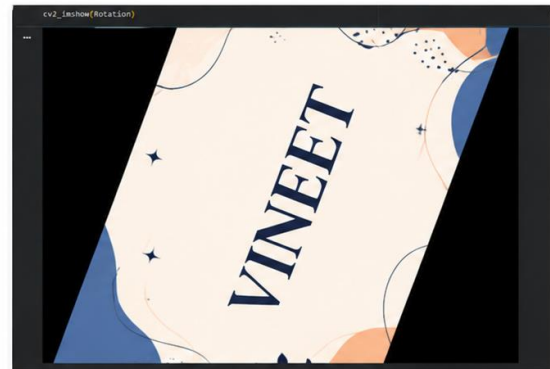
image = cv2.imread("/content/sample_data/Jinal(07).jpg")

#perform linear and cubic
image_re_linear = cv2.resize(image, None, fx=1.5,fy=2.5, interpolation=cv2.INTER_LINEAR)
image_re_cubic = cv2.resize(image , None, fx=1.5,fy=1.5, interpolation=cv2.INTER_CUBIC)

# Perform the translation
matrix = np.float32([[1, 0, 80], [0, 1, 80]])
translated = cv2.warpAffine(image, matrix, (image.shape[1], image.shape[0]))

#Rotation
height, width = image.shape[:2]
matrix1 = cv2.getRotationMatrix2D((width/2,height/2) , 70 , 1)
Rotation = cv2.warpAffine(image, matrix1 , (width , height))
cv2_imshow(image)

#Display the translated image
cv2_imshow(Rotation)
cv2_imshow(translated)
cv2_imshow(image_re_cubic)
cv2_imshow(image_re_linear)
```

OUTPUT-

Aim- Perform Image Stitching.

[illegible]

CODE:

```
# python -m pip install pillow
from PIL import Image
from IPython.display import display

# Read the three images
image1 = Image.open('/content/sample_data/Jinal(07).jpg')
image2 = Image.open('/content/sample_data/Middle_name.jpg')
image3 = Image.open('/content/sample_data/Surname.jpg')

# Show the original images image1.show() image2.show() image3.show()

# Resize the first image
image1 = image1.resize((426, 600))
image1_size = image1.size

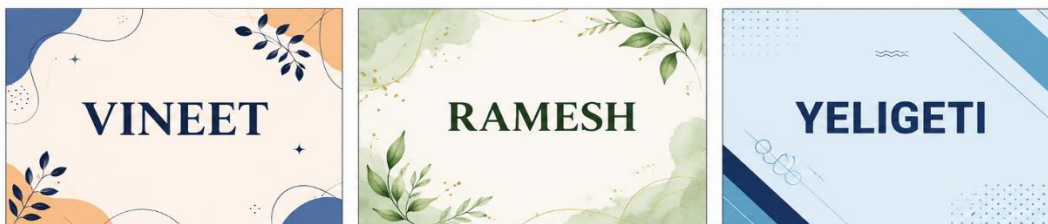
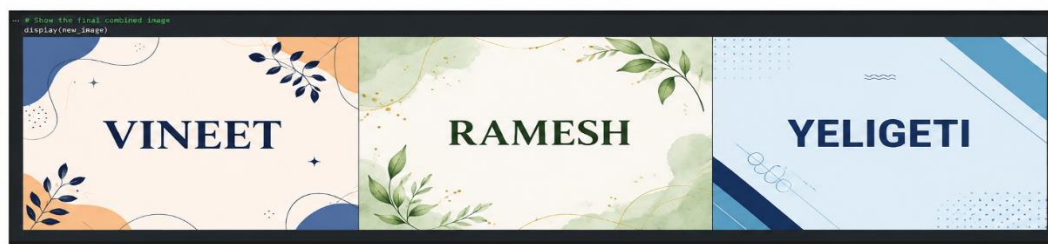
# Resize the third image to match the height of the first image
image3 = image3.resize((426, 600))

# Get the sizes of the images
image2_size = image2.size
image3_size = image3.size

# Create a new image with enough space for all three images side by side
new_image = Image.new('RGB', (image1_size[0] + image2_size[0] + image3_size[0], image1_size[1]), (250, 250, 250))

# Paste the images into the new image
new_image.paste(image1, (0, 0))
new_image.paste(image2, (image1_size[0], 0))
new_image.paste(image3, (image1_size[0] + image2_size[0], 0))

# Show the final combined image
display(new_image)
```

OUTPUT-**BEFORE IMAGE STICHING:****AFTER IMAGE STICHING:**

Aim- Perform Camera Calibration.

[illegible]

CODE:

```
import cv2
import numpy as np
import os
import glob
import matplotlib.pyplot as plt # Import matplotlib

# Defining the dimensions of checkerboard
CHECKERBOARD = (6, 9)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# Creating vector to store vectors of 3D points for each checkerboard image
objpoints = []
# Creating vector to store vectors of 2D points for each checkerboard image
imgpoints = []

# Defining the world coordinates for 3D points
objp = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.float32)
objp[0, :, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.reshape(-1, 2)
prev_img_shape = None

# Extracting path of individual image stored in a given directory
images = glob.glob('/content/sample_data/chess.jpeg')
for fname in images:
    img = cv2.imread(fname)
    if img is None:
        print(f"Error: Image at {fname} could not be loaded.")
        continue

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Find the chess board corners
    # If desired number of corners are found in the image then ret = true
    ret, corners = cv2.findChessboardCorners(gray, CHECKERBOARD,
                                              cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_FAST_CHECK +
                                              cv2.CALIB_CB_NORMALIZE_IMAGE)

    print(f"Found corners for {fname}: {ret}")

    # If desired number of corner are detected,
    # we refine the pixel coordinates and display
    # them on the images of checker board

    if ret == True:
        objpoints.append(objp)
        # refining pixel coordinates for given 2d points.
        corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)

        imgpoints.append(corners2)

        # Draw and display the corners
        img = cv2.drawChessboardCorners(img, CHECKERBOARD, corners2, ret)

# Display the image using matplotlib (suitable for Colab)
plt.figure(figsize=(10, 8))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title(f"Image: {os.path.basename(fname)} - Corners Found: {ret}")
```

```
plt.axis('off')
plt.show()

# cv2.destroyAllWindows() # Not needed with matplotlib

# Check if img was processed and has shape before proceeding
if 'img' in locals() and img is not None:
    h, w = img.shape[:2]
else:
    print("No valid image was processed. Cannot proceed with calibration.")
    h, w = (0, 0) # Assign dummy values to avoid error if no image processed

# Only attempt calibration if corners were found for at least one image
if len(objpoints) > 0:
    """
    Performing camera calibration by
    passing the value of known 3D points (objpoints)
    and corresponding pixel coordinates of the
    detected corners (imgpoints)
    """
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, gray.shape[:-1], None, None)

    print("Camera matrix : \n")
    print(mtx)
    print("dist : \n")
    print(dist)
    print("rvecs : \n")
    print(rvecs)
    print("tvecs : \n")
    print(tvecs)
else:
    print("Calibration skipped: No chessboard corners were detected in any image.")
```

OUTPUT:
ORIGINAL IMAGE



AFTER CORNER DETECTION



CODE:

```
import cv2
# Load the pre-trained face detector
face_detector = cv2.CascadeClassifier('/content/sample_data/haarcascade_frontalface_default.xml')

# Read the input image with the correct path
image = cv2.imread('/content/sample_data/MYPHOTO.jpg')

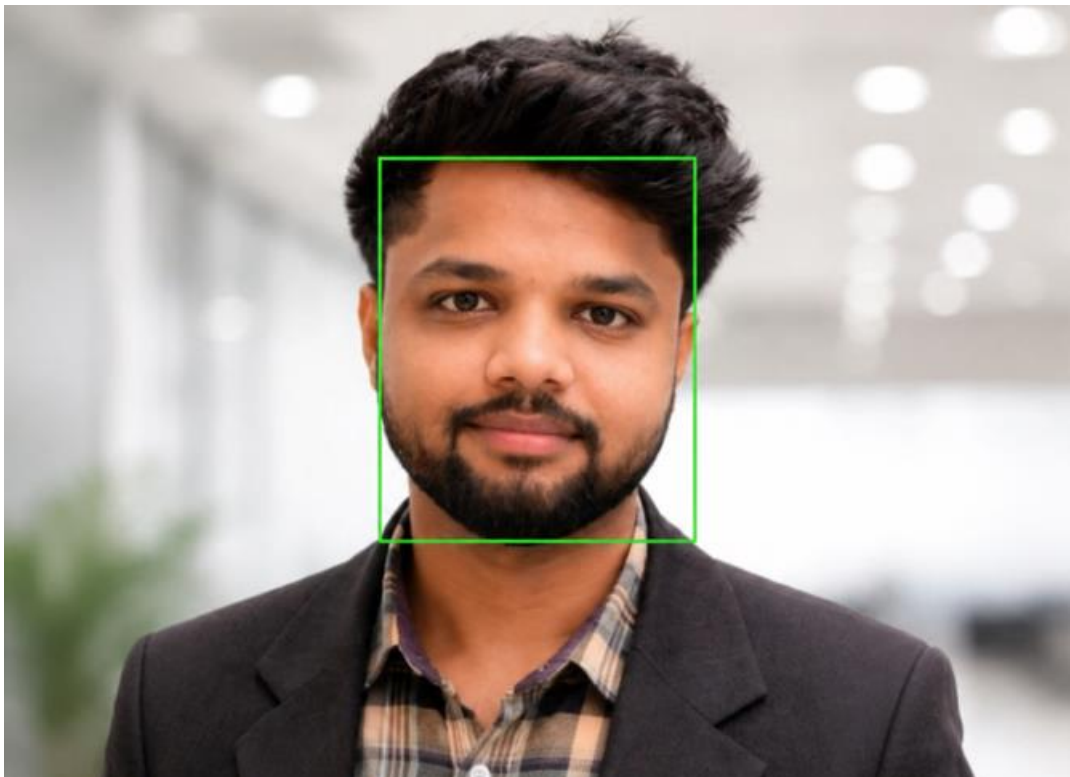
# Check if the image was loaded successfully
if image is None:
    print('Error: Could not load image.')
else:
    # Convert the image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Detect faces in the image
    faces = face_detector.detectMultiScale(gray, 1.3, 5)

    # Draw bounding boxes around the detected faces
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 5)

    cv2.imwrite('face_detected_image.jpg', image)
    print('Image with detected faces saved as face_detected_image.jpg')

import matplotlib.pyplot as plt
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.show()
```

OUTPUT:

[illegible]

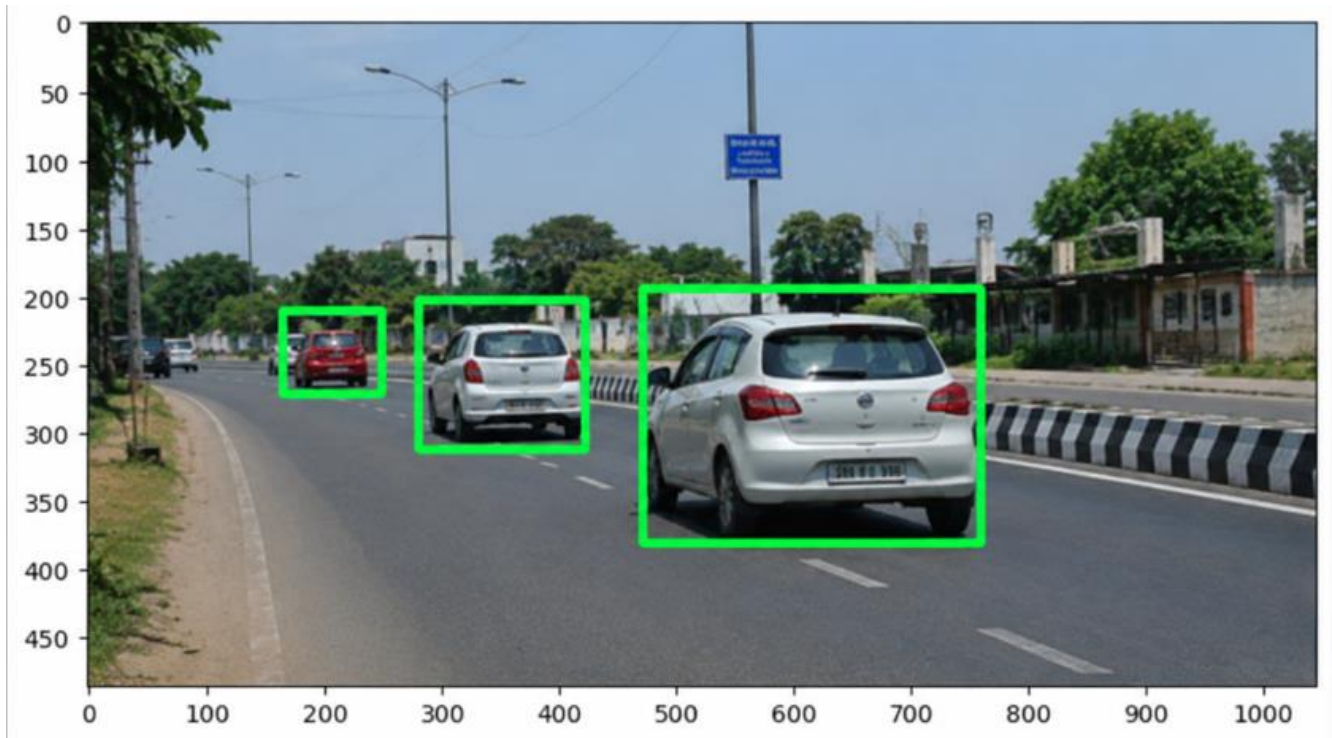
CODE:

```
import cv2
from matplotlib import pyplot as plt
# Paths
image_path = "/content/sample_data/car.jpeg"
cascade_path = '/content/sample_data/cars.xml'
# Load image
img = cv2.imread(image_path)
if img is None:
    raise FileNotFoundError("Image file could not be loaded. Check the path.")
# Convert image
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Load cascade
stop_data = cv2.CascadeClassifier(cascade_path)

if stop_data.empty():
    raise FileNotFoundError("Cascade file could not be loaded. Check the path.")

# Detect and draw rectangles
for (x, y, w, h) in stop_data.detectMultiScale(img_gray, minSize=(20, 20)):
    cv2.rectangle(img_rgb, (x, y), (x + h, y + w), (0, 255, 0), 5)
# Display image
plt.imshow(img_rgb)
plt.show()
```

OUTPUT:



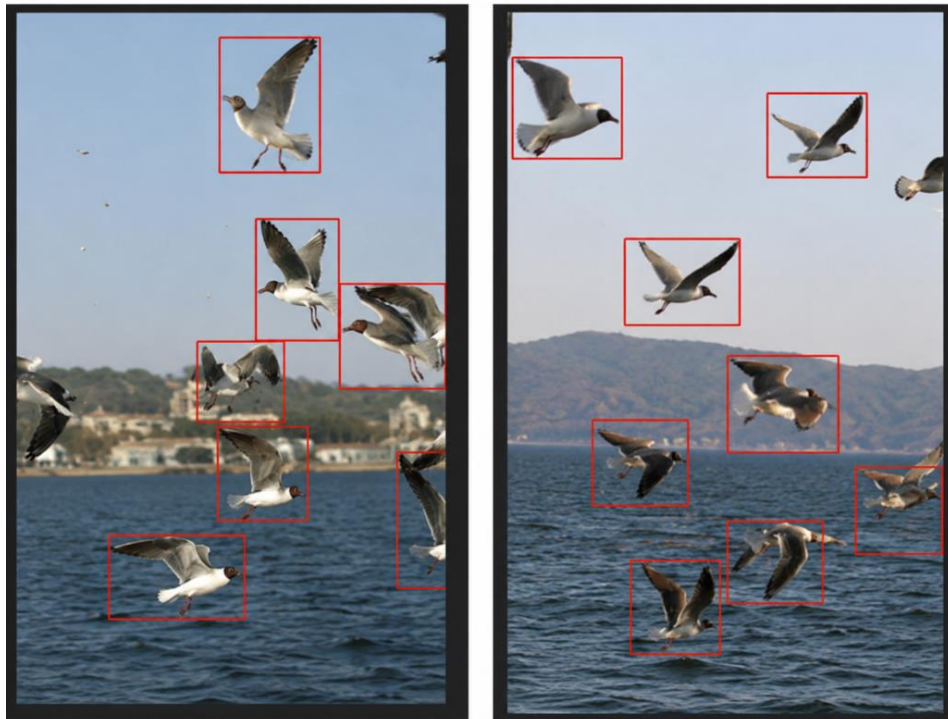
[illegible]

CODE

```
# pip install imutils
import cv2
import imutils
from google.colab.patches import cv2_imshow

# Initializing the HOG person detector
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

# Video capture from file
cap = cv2.VideoCapture('/content/sample_data/birds.mp4')
while cap.isOpened():
    ret, image = cap.read()
    if not ret:
        break
    image = imutils.resize(image, width=min(400, image.shape[1]))
    regions, _ = hog.detectMultiScale(image, winStride=(4, 4), padding=(4, 4), scale=1.05)
    for (x, y, w, h) in regions:
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 0, 255), 2)
    cv2_imshow(image)
    if cv2.waitKey(25) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

OUTPUT:

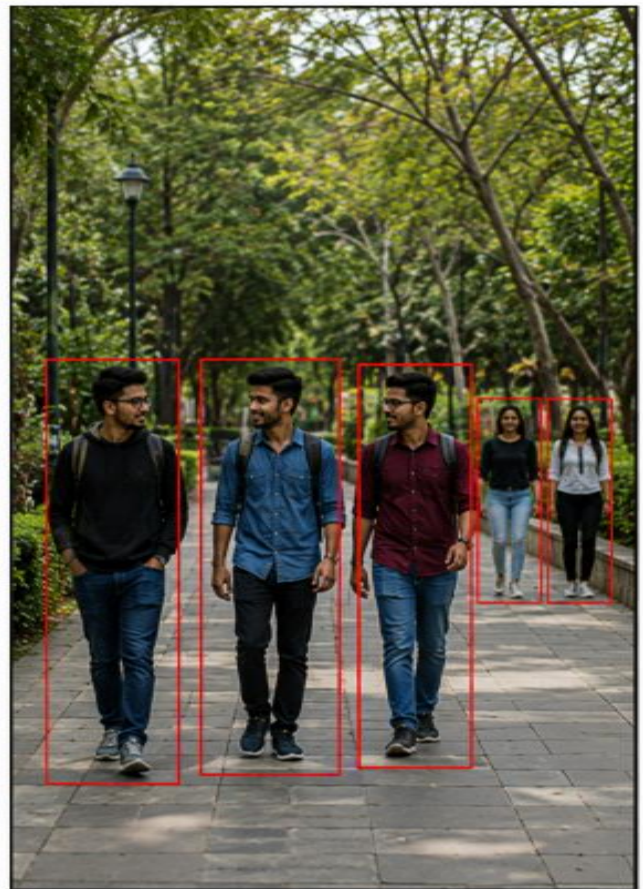
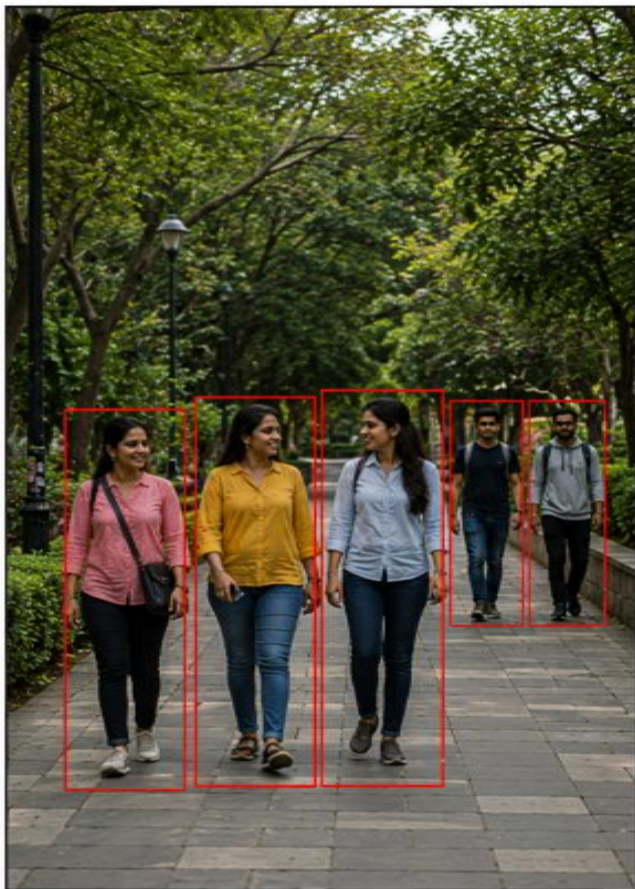
[illegible]

CODE:

```
# pip install imutils
import cv2
import imutils
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
cap = cv2.VideoCapture('/content/sample_data/face1.mp4')
while cap.isOpened():
    ret, image = cap.read()
    if not ret:
        break
    image = imutils.resize(image, width=min(400, image.shape[1]))
    regions, _ = hog.detectMultiScale(image, winStride=(4, 4), padding=(4, 4), scale=1.05)
    for (x, y, w, h) in regions:
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 0, 255), 2)
        cv2.imshow("Image", image)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break

# Release video capture and close all windows
cap.release()
cv2.destroyAllWindows()
```

OUTPUT:



Aim- Construct 3D model from images.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

CODE:

```
from PIL import Image
import numpy as np
from IPython.display import display # Import display for Colab

def shift_image(img, depth_img, shift_amount=10):
    # Ensure base image has alpha
    img = img.convert("RGBA")
    data = np.array(img)
    img_height, img_width, _ = data.shape # Get dimensions of the base image

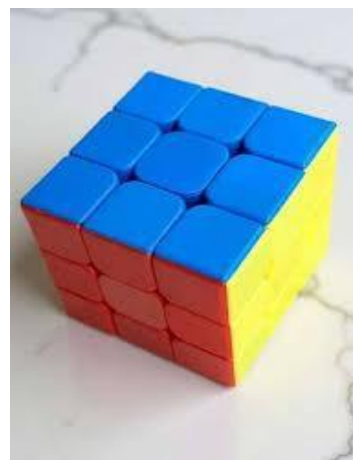
    depth_img = depth_img.convert("L")

    # Resize depth_img to match img dimensions if they are different
    if depth_img.size != (img_width, img_height):
        depth_img = depth_img.resize((img_width, img_height), Image.Resampling.LANCZOS) # Use a
        resampling filter

    depth_data = np.array(depth_img)
    deltas = ((depth_data / 255.0) * float(shift_amount)).astype(int)
    shifted_data = np.zeros_like(data)

    # Iterate over the dimensions of the original image (and thus shifted_data and deltas)
    for y in range(img_height):
        for x in range(img_width):
            dx = deltas[y, x] # Get the shift amount for the current pixel
            # Apply shift, checking bounds for the target column
            if 0 <= x + dx < img_width:
                shifted_data[y, x + dx] = data[y, x]
    shifted_image = Image.fromarray(shifted_data.astype(np.uint8))
    return shifted_image

img = Image.open("/content/sample_data/cube1.jpeg")
depth_img = Image.open("/content/sample_data/cube2.jpeg")
shifted_img = shift_image(img, depth_img, shift_amount=10)
display(shifted_img)
```

INPUT Images:

OUTPUT Model:



Aim- Implement object detection and tracking from video.

[illegible]

CODE:

```
import cv2
from google.colab.patches import cv2_imshow
from IPython.display import display
import numpy as np

# Load pre-trained YOLO model for object detection
net = cv2.dnn.readNet("/content/sample_data/yolov3.weights", "/content/sample_data/yolov3.cfg")
classes = []
with open("/content/sample_data/coco.names", "r") as f:
    classes = [line.strip() for line in f.readlines()]
layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
colors = [(0, 255, 0)] # Green color for bounding boxes

# Initialize video capture
cap = cv2.VideoCapture("/content/sample_data/12021304_1080_1920_60fps.mp4")

# Object tracking variables
tracked_objects = {}
next_id = 1

frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        print("End of video stream or error reading frame.")
        break
    frame_count += 1
    print(f"Processing frame {frame_count}...")

    height, width, channels = frame.shape

    # Object detection
    blob = cv2.dnn.blobFromImage(frame, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
    net.setInput(blob)
    outs = net.forward(output_layers)

    boxes = []
    confidences = []
    class_ids = []

    for out in outs:
        for detection in out:
            scores = detection[5:]
            class_id = scores.argmax()
            confidence = scores[class_id]
            if confidence > 0.5: # Reverted to standard confidence threshold
                center_x = int(detection[0] * width)
                center_y = int(detection[1] * height)
                w = int(detection[2] * width)
                h = int(detection[3] * height)

                x = int(center_x - w / 2)
```

```

        y = int(center_y - h / 2)

        boxes.append([x, y, w, h])
        confidences.append(float(confidence))
        class_ids.append(class_id)

print(f" Detections before NMS for frame {frame_count}: {len(boxes)}")
# Reverted NMS thresholds to standard values
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)

num_detections_after_nms = len(indexes.flatten()) if indexes is not None and len(indexes) > 0 else 0
print(f" Detections after NMS for frame {frame_count}: {num_detections_after_nms}")

# Object tracking and drawing bounding boxes
if indexes is not None and len(indexes) > 0:
    # Ensure indexes is a numpy array before flattening
    if not isinstance(indexes, np.ndarray):
        indexes = np.array(indexes)
    for i in indexes.flatten():
        x, y, w, h = boxes[i]
        label = str(classes[class_ids[i]])
        color = colors[0]

        # Check if object is already tracked
        matched_id = None
        for obj_id, tracked_box in tracked_objects.items():
            if abs(x - tracked_box[0]) < 50 and abs(y - tracked_box[1]) < 50:
                matched_id = obj_id
                break

        # Update or create new tracked object
        if matched_id is not None:
            tracked_objects[matched_id] = (x, y)
            cv2.putText(frame, f"{label} {matched_id}", (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
color, 2)
            cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)
        else:
            tracked_objects[next_id] = (x, y)
            cv2.putText(frame, f"{label} {next_id}", (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color,
2)
            cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)
            next_id += 1

cv2.imshow(frame)

cap.release()
print("Video processing finished.")

```

OUTPUT:



Aim- Perform Feature extraction using RANSAC.

[illegible]

CODE:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

def extract_features_and_match(img1, img2):
    # Initiate SIFT detector
    sift = cv2.SIFT_create()

    # Find the keypoints and descriptors with SIFT
    kp1, des1 = sift.detectAndCompute(img1, None)
    kp2, des2 = sift.detectAndCompute(img2, None)

    # BFMatcher with default params
    bf = cv2.BFMatcher()
    matches = bf.knnMatch(des1, des2, k=2)

    # Apply ratio test
    good_matches = []
    for m, n in matches:
        if m.distance < 0.75 * n.distance:
            good_matches.append(m)

    return kp1, kp2, good_matches

def apply_ransac(kp1, kp2, matches):
    # Extract corresponding points
    src_pts = np.float32([kp1[m.queryIdx].pt for m in matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in matches]).reshape(-1, 1, 2)

    # Find homography matrix using RANSAC
    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

    return M, mask

def draw_matches_with_mask(img1, kp1, img2, kp2, matches, mask):
    matches_mask = mask.ravel().tolist()
    draw_params = dict(matchColor=(0, 255, 0), # draw matches in green color
                        singlePointColor=None,
                        matchesMask=matches_mask, # draw only inliers
                        flags=2)
    img_matches = cv2.drawMatches(img1, kp1, img2, kp2, matches, None, **draw_params)
    return img_matches

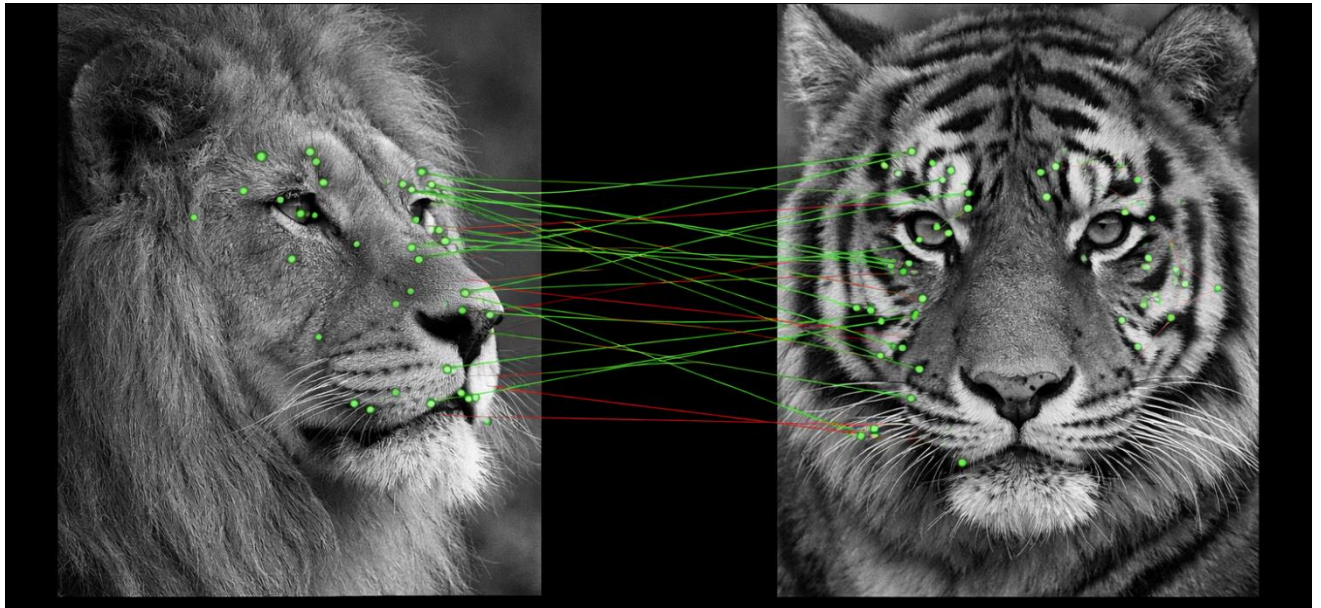
# Load images
img1 = cv2.imread('/content/sample_data/lion.jpeg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('/content/sample_data/tiger.jpeg', cv2.IMREAD_GRAYSCALE)
# Extract features and match
kp1, kp2, matches = extract_features_and_match(img1, img2)

# Apply RANSAC
M, mask = apply_ransac(kp1, kp2, matches)

# Draw matches with inliers highlighted
img_matches = draw_matches_with_mask(img1, kp1, img2, kp2, matches, mask)
```

```
# Display the result  
cv2.imshow('img_matches',  
img_matches)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT:



Aim- Perform Colorization.

[illegible]

CODE:

```

import numpy as np
import cv2
from cv2 import dnn

# Model file paths
proto_file = '/content/sample_data/colorization_deploy_v2.prototxt'
model_file = '/content/sample_data/colorization_release_v2 (2).caffemodel'
hull_pts = '/content/sample_data/pts_in_hull.npy'
img_path = '/content/sample_data/paris.jpeg'

img = cv2.imread(img_path)

if img is None:
    print(f"Error: Unable to load image at {img_path}. Please make sure the image file exists.")
else:
    try:
        scaled = img.astype("float32") / 255.0
        lab_img = cv2.cvtColor(scaled, cv2.COLOR_BGR2LAB)

        net = dnn.readNetFromCaffe(proto_file, model_file)
        kernel = np.load(hull_pts)

        class8 = net.getLayerId("class8_ab")
        conv8 = net.getLayerId("conv8_313_rh")
        pts = kernel.transpose().reshape(2, 313, 1, 1)
        net.getLayer(class8).blobs = [pts.astype("float32")]
        net.getLayer(conv8).blobs = [np.full([1, 313], 2.606, dtype="float32")]

        resized = cv2.resize(lab_img, (224, 224))
        L = cv2.split(resized)[0]
        L -= 50

        net.setInput(cv2.dnn.blobFromImage(L))
        ab_channel = net.forward()[0, :, :, :].transpose((1, 2, 0))
        ab_channel = cv2.resize(ab_channel, (img.shape[1], img.shape[0]))

        # Convert to colorized image in BGR
        L = cv2.split(lab_img)[0]

        colorized = cv2.cvtColor(np.concatenate((L[:, :, np.newaxis], ab_channel), axis=2),
cv2.COLOR_LAB2BGR)
        colorized = np.clip((255 * colorized).astype("uint8"), 0, 255)

        # Resize images for display
        img_display = cv2.resize(img, (640, 640))
        colorized_display = cv2.resize(colorized, (640, 640))

        # Concatenate and display images
        result = np.hstack([img_display, colorized_display])
        cv2.imshow("Grayscale -> Colour", result)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
    except cv2.error as e:
        print(f"Error loading Caffe model or protocol buffer: {e}")

```

```
print("Please ensure 'colorization_deploy_v2.prototxt' and 'colorization_release_v2.caffemodel' are valid  
and uncorrupted.")  
except Exception as e:  
    print(f"An unexpected error occurred: {e}")
```

OUTPUT:



Aim- Perform Text detection and recognition.

[illegible]

CODE:

```
import cv2
import pytesseract
import os

# Set the path to the Tesseract executable for Colab
pytesseract.pytesseract.tesseract_cmd = r'/usr/bin/tesseract'

# Set TESSDATA_PREFIX to point to the tessdata directory for installed languages
os.environ['TESSDATA_PREFIX'] = r'/usr/share/tesseract-ocr/4.00/tessdata/'

image_path = '/content/sample_data/prac9.jpg'
img = cv2.imread(image_path)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
ret, thresh1 = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU | cv2.THRESH_BINARY_INV)
rect_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (18, 18))
dilation = cv2.dilate(thresh1, rect_kernel, iterations=1)
contours, hierarchy = cv2.findContours(dilation, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
im2 = img.copy()
output_path = '/content/sample_data/recognized.txt'
with open(output_path, "w+") as file:
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        cropped = im2[y:y + h, x:x + w]
        text = pytesseract.image_to_string(cropped)
        print(f'Recognized text: {text}')
        file.write(text + "\n")
```

OUTPUT:

WAITING?

PLEASE

TURN OFF

YOUR

ENGINE

Aim- Perform Image matting and compositing.

[illegible]

CODE:

```
import matplotlib.pyplot as plt
from PIL import Image
import requests

# Load the image and trimap
url = "https://github.com/hustvl/ViTMatte/blob/main/demo/bulb_rgb.png?raw=true"
image = Image.open(requests.get(url, stream=True).raw).convert("RGB")
url = "https://github.com/hustvl/ViTMatte/blob/main/demo/bulb_trimap.png?raw=true"
trimap = Image.open(requests.get(url, stream=True).raw)

# Display the image and trimap
plt.figure(figsize=(15, 15))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.subplot(1, 2, 2)
plt.imshow(trimap)
plt.show()
```

OUTPUT:

[illegible]

CODE:

```
from PIL import Image
from IPython.display import display # Import display

# Load images and convert to mode 'L'
im1 = Image.open('/content/sample_data/car.jpeg').convert('L')
im2 = Image.open('/content/sample_data/nature.webp').convert('L')
mask = Image.open('/content/sample_data/nature1.webp').convert('L')

# Ensure all images are the same size
size = im1.size
im2 = im2.resize(size)
mask = mask.resize(size)

# Composite the images
im3 = Image.composite(im1, im2, mask)

# Show the result using display() for Colab output
display(im3)
```

OUTPUT: